

ia-26/a_star.py

```

1 import numpy as np
2 import math
3
4 class Node:
5
6     def __init__(self, position):
7         self.position = position
8
9         # Custos
10
11         self.g = np.inf
12         self.h = 0
13         self.f = np.inf
14
15         self.parent = None
16
17 # Distância Euclidiana
18 def heuristic(current, goal):
19
20     return math.sqrt(
21         (current[0] - goal[0])**2
22         +
23         (current[1] - goal[1])**2
24     )
25
26 # Reconstrução do caminho
27 def reconstruct_path(node):
28     path = []
29     current = node
30
31     while current is not None:
32         path.append(current.position)
33         current = current.parent
34
35     return path[::-1]
36
37
38 def a_star(grid, start, goal):
39     rows = len(grid)
40     cols = len(grid[0])
41
42     # Cria matriz de nós
43     nodes = [
44
45         [Node((i, j)) for j in range(cols)]
46
47         for i in range(rows)
48     ]
49
50     # Nó inicial

```

```

51     start_node = nodes[start[0]][start[1]]
52     start_node.g = 0
53     start_node.h = heuristic(
54         start,
55         goal
56     )
57     start_node.f = (
58         start_node.g
59         + start_node.h
60     )
61
62     # Listas
63     open_list = [start_node]
64     closed_list = []
65
66     # Movimentos
67     directions = [
68         (-1, 0), # cima
69         (1, 0), # baixo
70         (0, -1), # esquerda
71         (0, 1) # direita
72     ]
73
74
75     while len(open_list) > 0:
76         # Remove menor F
77         current_node = min(
78             open_list,
79             key=lambda node: node.f
80         )
81
82         open_list.remove(current_node)
83
84         closed_list.append(
85             current_node.position
86         )
87
88         # Destino
89         if current_node.position == goal:
90
91             return reconstruct_path(
92                 current_node
93             ), current_node.g
94
95     # Expansão
96     for direction in directions:
97         row = (
98             current_node.position[0]
99             + direction[0]
100         )
101         col = (
102             current_node.position[1]
103             + direction[1]

```

```

104 )
105
106 # Limites
107 if (
108     row < 0
109     or row >= rows
110     or col < 0
111     or col >= cols
112 ):
113
114     continue
115
116 # Obstáculo
117 if grid[row][col] == "*":
118     continue
119
120 # Fechada
121 if (row, col) in closed_list:
122     continue
123
124 neighbor = nodes[row][col]
125
126 # Custo da ação
127 cell_cost = grid[row][col]
128
129 # G = Ganterior + custo
130 g_new = (
131     current_node.g
132     + cell_cost
133 )
134
135 h_new = heuristic(
136     (row, col),
137     goal
138 )
139
140 f_new = g_new + h_new
141
142 # Atualiza se melhor
143 if f_new < neighbor.f:
144
145     neighbor.g = g_new
146
147     neighbor.h = h_new
148
149     neighbor.f = f_new
150
151     neighbor.parent = current_node
152
153     if neighbor not in open_list:
154
155         open_list.append(
156             neighbor

```

```

157 )
158
159 return None, np.inf
160
161
162 # Exemplo
163
164 if __name__ == "__main__":
165
166     grid = [
167
168         ["I", 2, 1, 1, 1, 2],
169         [1, "*", 2, "*", 3, 1],
170         [1, 4, 15, "*", "*", 1],
171         [2, "*", 15, "*", 4, 1],
172         ["*", "*", 2, 2, 9, 1],
173         [2, 1, 1, 1, "F", 1]
174     ]
175
176 # Conversão
177 start = None
178 goal = None
179
180
181 for i in range(len(grid)):
182     for j in range(len(grid[0])):
183         if grid[i][j] == "I":
184             start = (i, j)
185             grid[i][j] = 0
186         elif grid[i][j] == "F":
187             goal = (i, j)
188             grid[i][j] = 0
189
190 # Executa
191 path, cost = a_star(
192     grid,
193     start,
194     goal
195 )
196
197 # Resultado
198 print("\n===== CAMINHO =====\n")
199 for p in path:
200     print(p)
201 print("\nCusto total:")
202 print(cost)

```